

Software-Lifecycle-Management

Dokumentation

Inhalt

1	Organization Kanban-Board – Branches – Issues - Commits.....	2
1.1	Link to Git-Hub Project	2
1.2	Items.....	2
1.3	Branches	2
1.4	Issue (in den Repositories)	2
1.5	Commit message.....	3
2	Festlegungen GitHub Repos	3
2.1	Link to Git-Hub Repository.....	3
2.2	Einstellungen*	3
2.3	Regeln	3
3	Workflow	4
3.1	Erstellen von Issues (Items).....	4
3.1.1	Issues anlegen (über Repository)	5
3.1.2	Issues anlegen (über Projekt).....	5
3.1.3	Issues in ein Repository verschieben	6
3.2	Sub-Issues erstellen	7
3.3	Arbeiten mit Status	8
3.4	Branches	8
3.5	Commits	9
3.6	Review.....	9
3.7	Testing.....	10
3.7.1	Unit Tests	10
3.7.2	CI Workflow	11
3.8	Deployment.....	12
3.8.1	CD Workflow.....	12
3.8.2	Jar-Datei herunterladen.....	13
3.8.3	Jar-Datei lokal ausführen.....	13

1 Organization Kanban-Board – Branches – Issues – Commits

1.1 Link to Git-Hub Project

<https://github.com/users/Pfnei/projects/1>

Dem Projekt sind zwei Repositories zugeordnet – siehe: **LINK TO GIT-HUB REPOSITORY**

1.2 Items

Einschub: Als Items werden „Kanban-Cards“ in Github Project bezeichnet

Items sind als **Features** (main user story) **Task** und (**Subtasks**) organisiert.

Bezeichnung für die Features – selbe Bezeichnung wird für die Branches verwendet

- feature-1/make-game-move
- feature-2/see-game-state
- feature-3/game-end-notification
- feature-4/replay-game

Bezeichnung für Tasks und Subtasks – selbe Bezeichnung wird für die Issues (im Repo) verwendet - Elemente

- bugfix-/
- task-/
- refactor-/
- docu-/

Die Nummerierung ist über alle Elemente UND BEIDE Repos (nicht je Element) fortlaufend und beginnt mit 100, z.B.

- docu-100/start-documentation
- task-101/initial-project-setup

Tasks /Subtasks/Issues können (müssen aber nicht) Bezug zu anderen Items haben. Dann ist dies den Details zu vermerken bzw. für die Zuordnung von Subtasks siehe: **3.2 SUB-ISSUES ERSTELLEN**

1.3 Branches

Sollen dieselben Namen haben wie die **Features**. Es kann aber auch andere Branches geben, falls das inhaltlich Sinn macht, dann sollen sie wie Items/Issue heißen, z.B.

- task-103/ setup initial class structure

1.4 Issue (in den Repositories)

Sollen dieselben Namen haben wie die Tasks (Subtasks)

1.5 Commit message

Abkürzung als Referenz plus kurze Beschreibung – „fix, added, updated, changed, removed“ verwenden!

Es kann zu einer Referenzbezeichnung mehrere commit messages geben

Bsp:

- docu-100: started documentation
- docu-100: added new chapter sample project with screenshots
- bugfix-157: user login is working again

2 Festlegungen GitHub Repos

2.1 Link to Git-Hub Repository

https://github.com/Pfnei/SLM_TicTacToe (“Coding Repo”)

https://github.com/Pfnei/SLM_TicTacToe_Documentation.git (“Doku-Repo”)

Das eigene Repository wurde gewählt, um unkompliziert gleichzeitig am Code und der Dokumentation arbeiten zu können.

2.2 Einstellungen*

Folgende zusätzliche Einstellungen wurden für den Main Branch getroffen

- Require a pull request before merging
- Required approvals : #1
- Do not allow bypassing the above settings

2.3 Regeln

1. No direct commits to main
2. Every merge request needs one approval
3. Jar files müssen in jedem merge in den main branch erstellt werden
4. Tests müssen automatisch für jeden feature branch push ausgeführt werden

*Einstellungen gelten nur für das “Coding Repo”. Im “Doku-Repo” gibt es keine Regeln, es gibt auch nur einen Branch – main -, auf den auch direkt committed werden kann.

3 Workflow

Der Workflow wird nicht anhand eines Features (zu große Einheit) sondern anhand von Tasks beschrieben

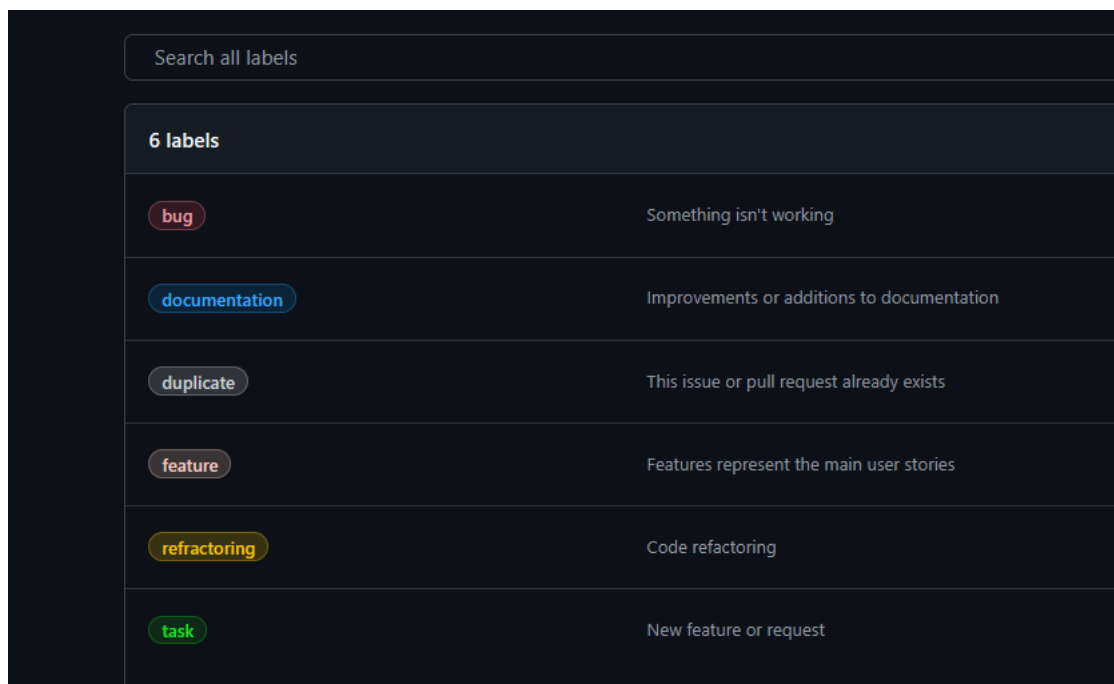
3.1 Erstellen von Issues (Items)

Issues / Items können entweder im Repository oder als Item im Project erstellt werden. Das Projekt ist standardmäßig mit dem „Coding Repo“ verbunden, dh. hier werden Issues und Items automatisch gegenseitig zugeordnet

Für das „Doku_Repo“ ist es wichtig, die Issues / Items direkt im PROJEKT zu erstellen (es wird vorher mit # das richtige Repository ausgewählt), da sonst die Issues nicht im Projekt aufscheinen.

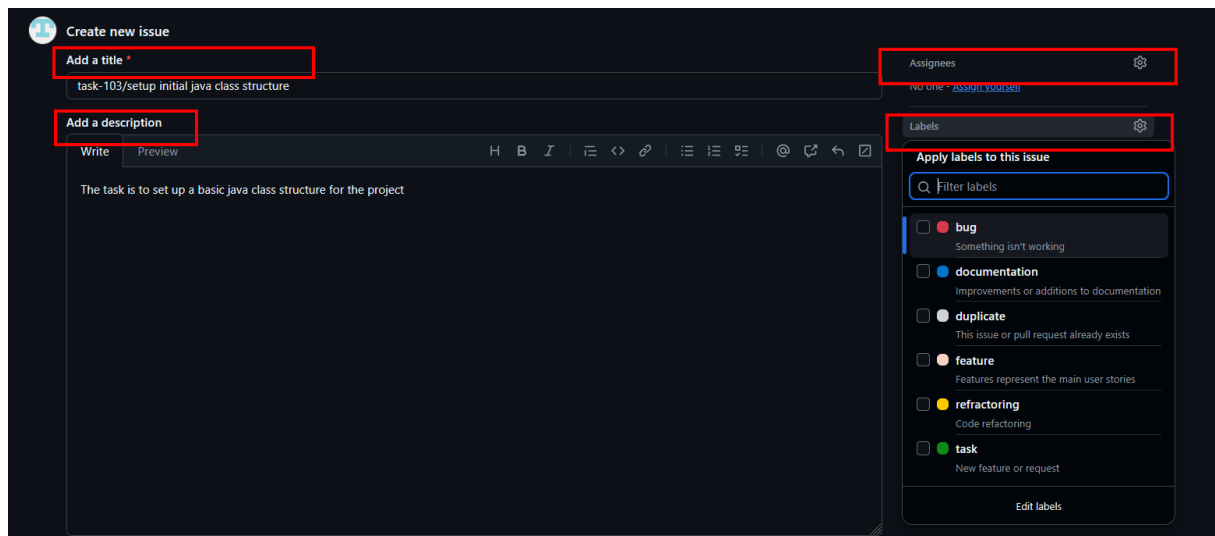
Das Issues kann entweder beim Erstellen einem (oder mehreren Personen) zugeordnet werden, oder es wird sich später „selbstständig“ vom Kanban-Baord genommen.

Die Issues bekommen auch Labels – die Labels sind ident mit den Bezeichnungen für Features und Tasks (zusätzliches gibt es noch ein „duplicate“ Label)



3.1.1 Issues anlegen (über Repository)

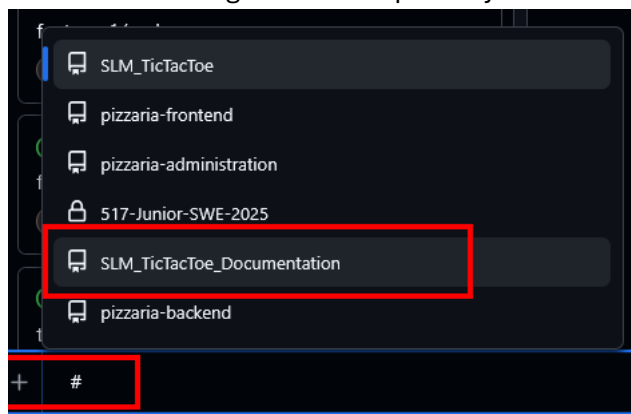
Macht nur bei beim „Coding Repo“ Sinn, da es automatisch mit dem Projekt verbunden ist
Issues -> New issue:

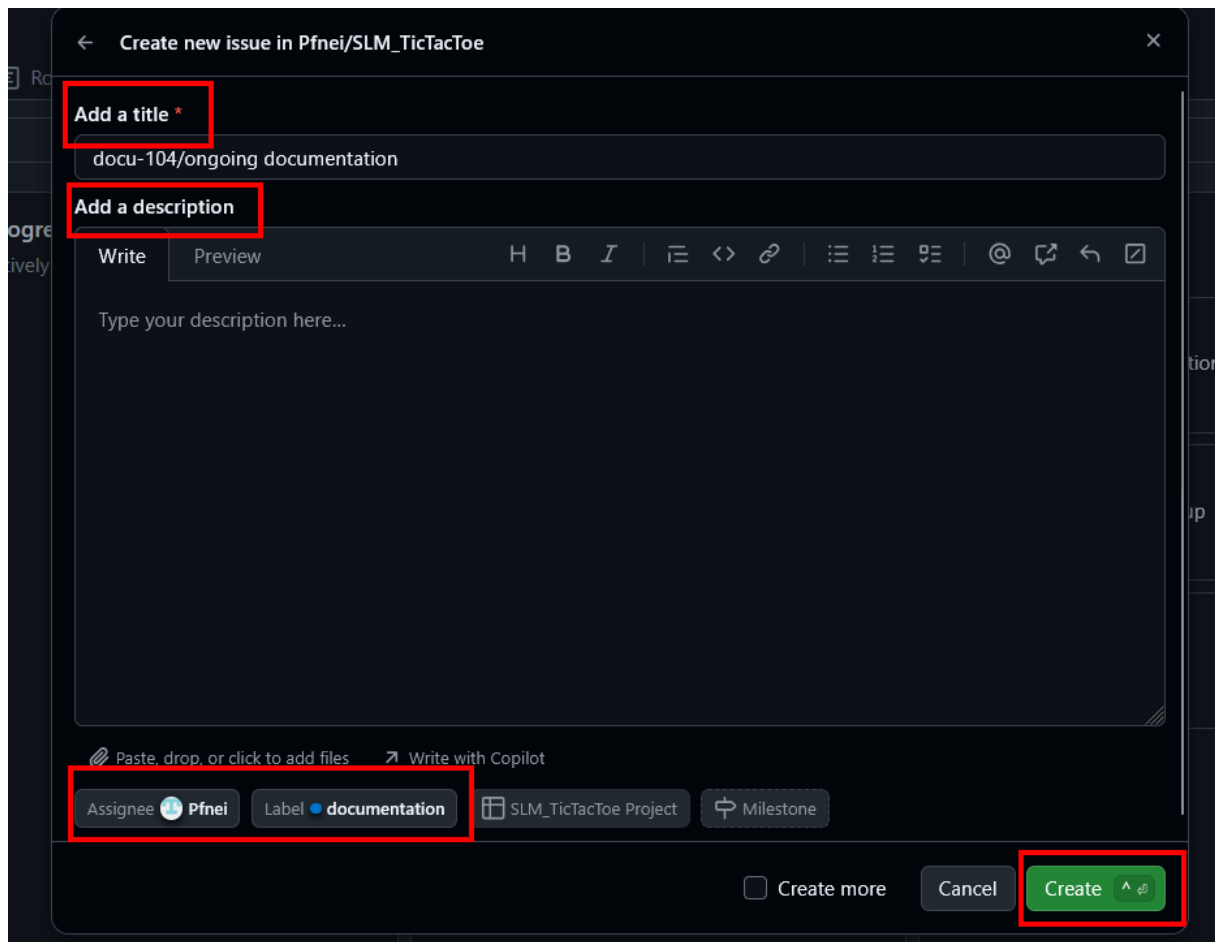


3.1.2 Issues anlegen (über Projekt)

Kann für beide Repos verwendet werden. ABER beim “Doku-Repo” ist vorher das Repo auszuwählen (mit # in der + Add item Leiste)

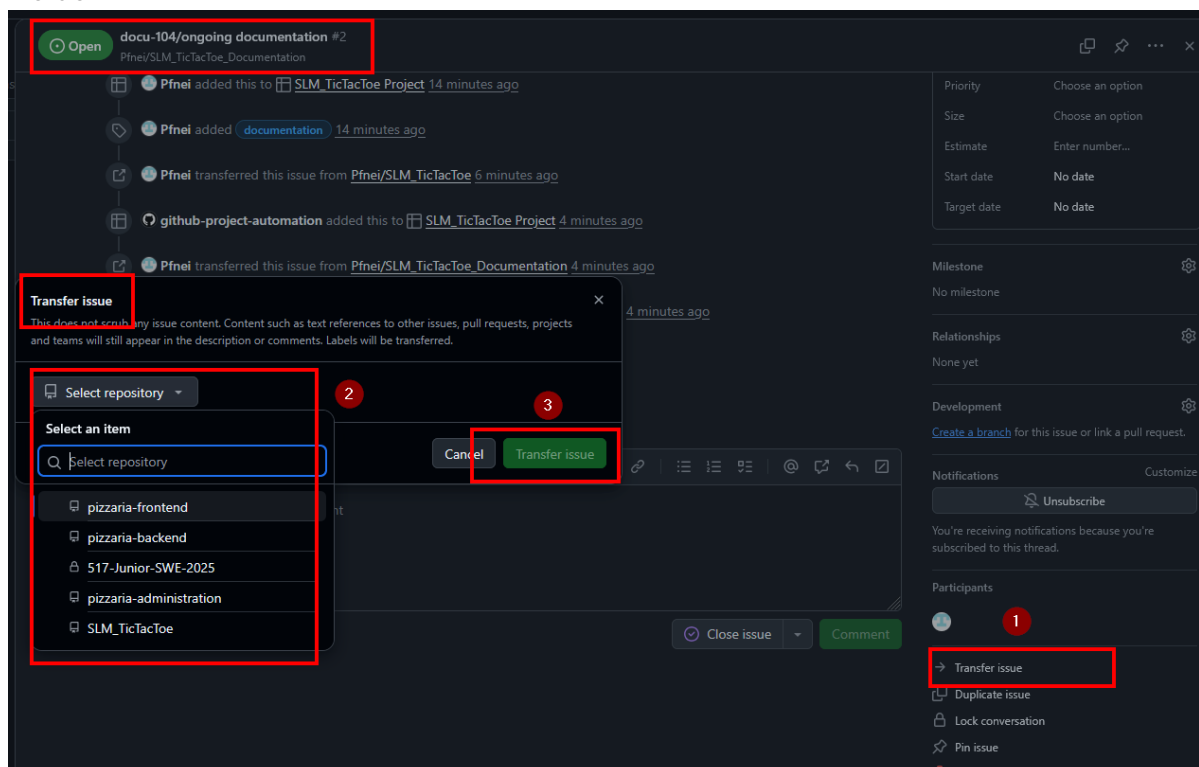
+ Add Item in ausgewähltes Repository:





3.1.3 Issues in ein Repository verschieben

Falls obiger Ansatz nicht funktioniert, können Issues auch in ein andere Repository verschoben werden:



3.2 Sub-Issues erstellen

Issues können im Projekt über Relationships als Sub-Issues einem anderen Issue zugeordnet werden.

The screenshot shows a Jira issue page for 'task-103/setup initial java class structure #8'. The 'Relationships' sidebar on the right is highlighted with a red box. It shows the 'Add parent' button (1) and the 'Mark as blocked by' button (2). The 'Parent issue' section shows 'feature-1/make-game-move' (3) with a '1 / 3' indicator. The main content area shows a list of recent activity, including 'Pfnei moved this from In progress to Done in SLM_TicTacToe Project' and 'Pfnei closed this as completed by moving to Done in SLM_TicTacToe P'.

Im Board erkennt man auf den Cards, wie viele Sub-Issues es zu einem Issue gibt, und wie viele davon schon auf “done” sind:

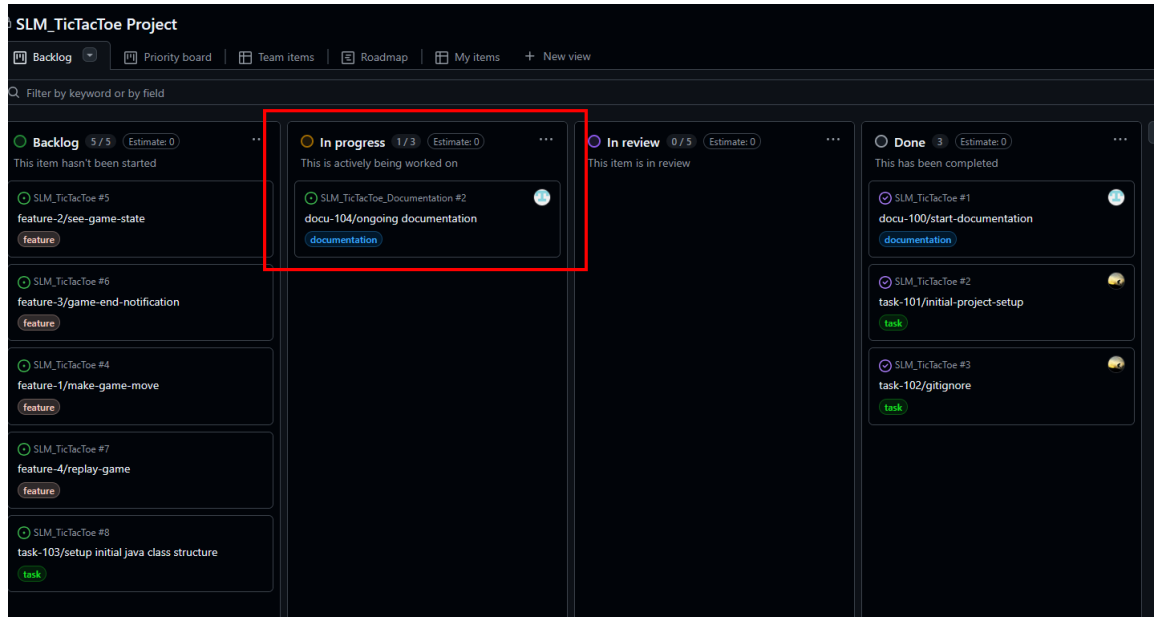
The screenshot shows a Jira project board for 'SLM_TicTacToe Project'. The board has columns for 'Backlog', 'In progress', and 'In'. The 'Backlog' column shows a card for 'SLM_TicTacToe #30' with the title 'task-117/add exit possibility for game'. The 'In progress' column shows a card for 'SLM_TicTacToe #7' with the title 'feature-4/replay-game'. This card is highlighted with a red box and shows a progress bar with '0 / 1' and '0%'.

3.3 Arbeiten mit Status

In unserem Projekt gibt es 4 Status: „Backlog“, „In progress“, „In review“, „Done“

Die Teammitglieder können selbstständig den Status einer Issue ändern – Issues die nicht mehr im Backlog sind, sollten zumindest einem Teammitglied zugeordnet sein.

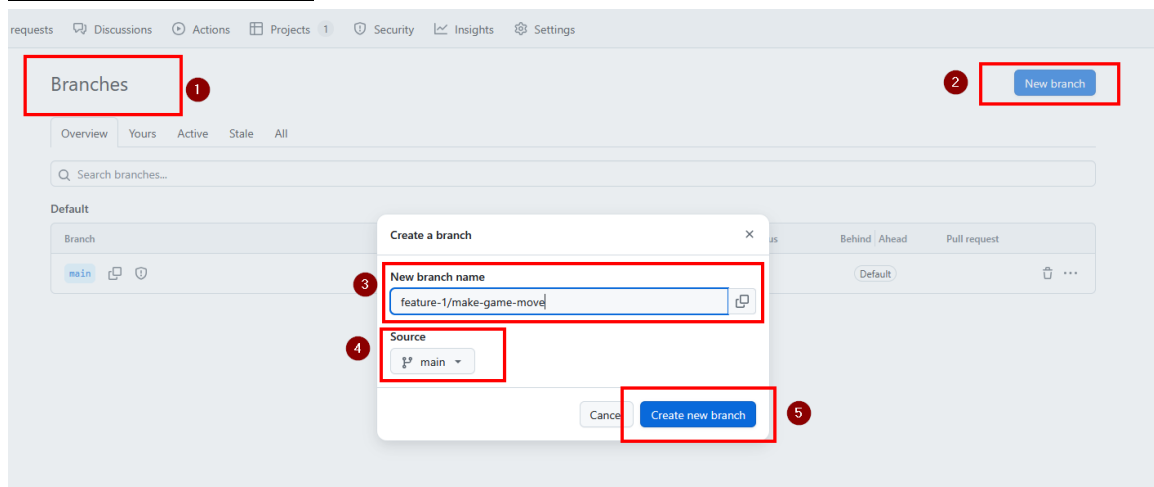
Bsp – Verschieben von „docu-104/ongoing documentation“ zu „in Progress“



3.4 Branches

Issues können entweder in bestehenden Branches abgearbeitet werden, oder es kann ein neuer Branch passend zum Issue erstellt werden.

Branches-> New branch



Lokal auf der Console auf main wechseln. pullen und in den neuen Branch wechseln

git checkout main

git pull (zum Downloaden der neuen Branch)

git checkout NEWBRANCH

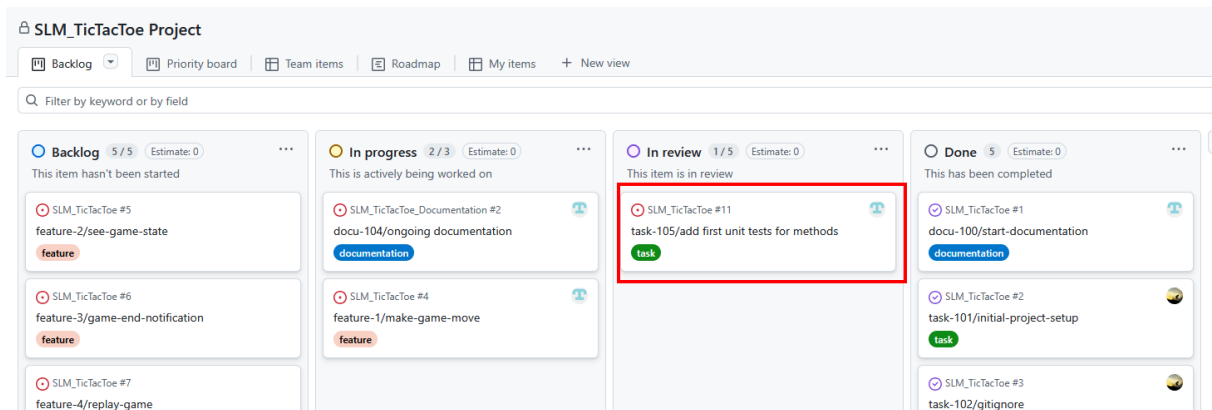
3.5 Commits

Commit-Messages haben diese Structur: **1.5 COMMIT MESSAGE:**

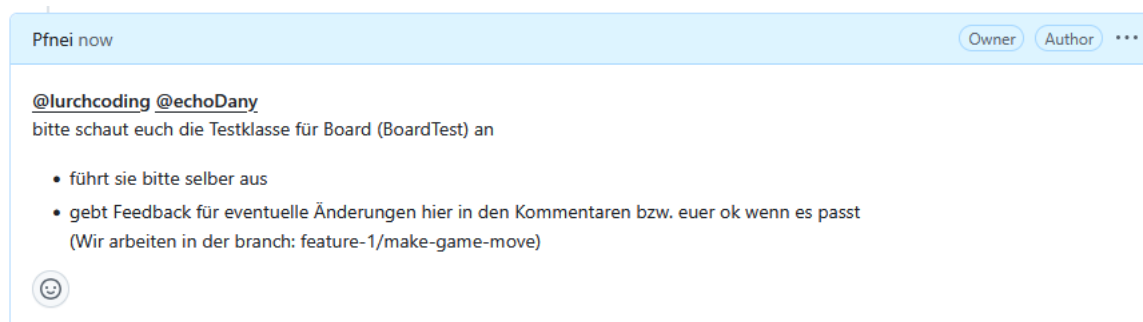
3.6 Review

Es muss NICHT jeder Commit reviewed werden. Zumindest vor jedem Merge in „main“ ist Review erforderlich. Der Review Prozess wird über GitHub Projects abgebildet

Issue nach „In review“ verschieben



Und über Kommentare mit den Teammitgliedern kommunizieren



3.7 Testing

3.7.1 Unit Tests

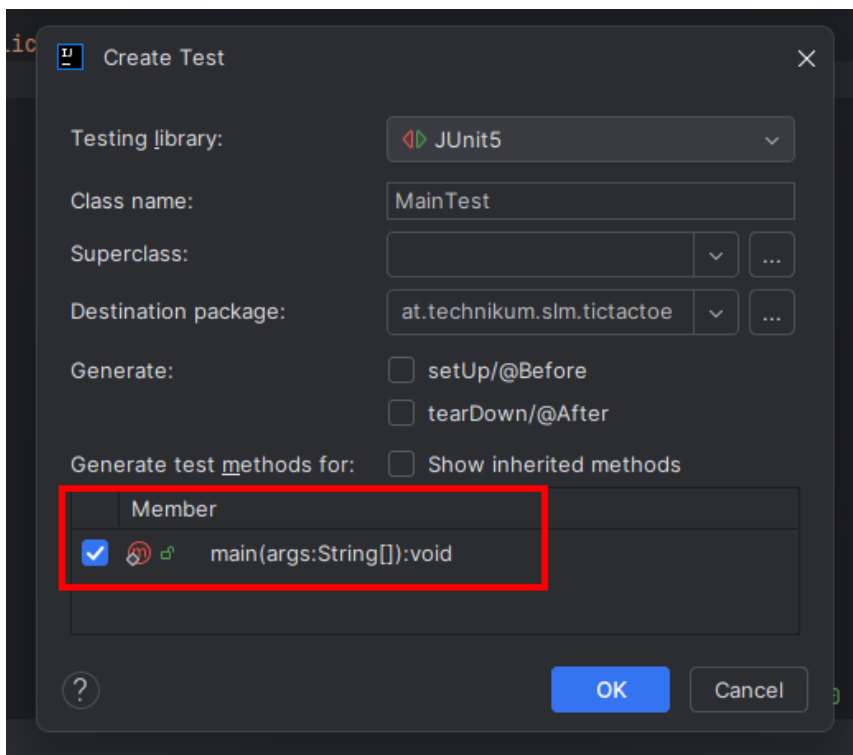
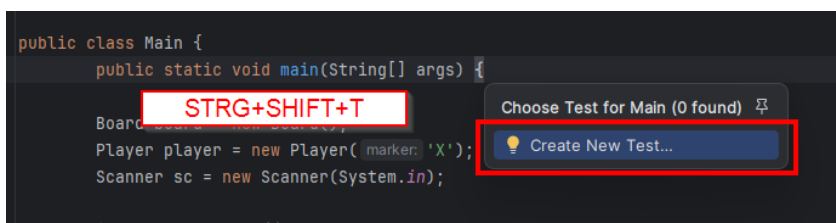
Pro Method sind mindestens zwei Unit Tests zu schreiben. Es sind positive und negative Testfälle zu schreiben.

Pro Klasse wird eine Testklasse erzeugt, Die Testmethoden können, wie die Methoden in der Klasse heißen (und dort mehrere Tests enthalten), oder auch spezifischer Namen haben und jeweils nur einen Test für diese Methode durchführen.

Einfache Erzeugung der Testklasse in IntelliJ:

In die Klasse stellen:

STRG+SHIFT+T → create new Test --> danach unbedingt die Members mitauswählen:



Bespiel: Klasse Board(Test) – Methode isEmpty:

```
ci.yml BoardTest.java x
1 package at.technikum.slm.tictactoe;
2
3 import org.junit.jupiter.api.Test;
4
5 import static org.junit.jupiter.api.Assertions.*;
6
7 class BoardTest {
8
9     @Test
10    void isEmpty() {
11        Board board = new Board();
12        assertTrue(board.isEmpty(x: 1, y: 2));
13
14        board.place(x: 0, y: 0, marker: '\0');
15        assertTrue(board.isEmpty(x: 0, y: 0));
16
17        board.place(x: 2, y: 2, marker: 'U');
18        assertFalse(board.isEmpty(x: 2, y: 2));
19
20        assertThrows(ArrayIndexOutOfBoundsException.class, () -> {
21            board.isEmpty(x: -2, y: 1);
22        });
23    }
24 }
```

Test können in der IDE (wir verwenden IntelliJ) direkt in der Testklasse ausgeführt werden („Run“-Button neben der Klasse oder den einzelnen Methoden).

Es können auch alle Tests des Projekts auf einmal in der IDE getestet werden:

1. Im Projektbaum auf **src/test/java** → **Rechtsklick**
2. **Run 'All Tests'**

3.7.2 CI Workflow

Der CI-Workflow ist über GitHub Actions abgebildet.

Test werden automatisch bei jedem Branch push (wenn er dem Bezeichnungsschema von **1.3 BRANCHES** entspricht) und bei jedem Pull Request auf „main“ ausgeführt.

Das Ergebnis unter Actions bei Auswahl des entsprechenden Workflows:

Actions-> Java CI with Maven

The screenshot shows the GitHub Actions interface. The 'Actions' tab is selected, and the 'Java CI with Maven' workflow is chosen from the left sidebar. The main area displays two workflow runs:

Run Name	Status	Event	Branch	Actor	Time
added maven cache to CI workflow	Completed	feature-1/make-game-move	feature-1/make-game-move	...	28 minutes ago, 33s
task-106: adapted branch paths in CI workflow	Completed	feature-1/make-game-move	feature-1/make-game-move	...	35 minutes ago, 20s

3.8 Deployment

Bei jedem Pull Request auf „main“ kommt es zu einem „Pseudo_Deployment“ – es wird eine downloadable jar.Datei erstellt.

3.8.1 CD Workflow

Der CD-Workflow ist über GitHub Actions abgebildet.

Bei jedem PUSH auf „main“ (kann nur durch eine Pull Request erfolgen, da direkte Commits auf main nicht erlaubt sind) wird eine jar.Datei erstellt.

Das Ergebnis ist entweder im Pull Request oder unter Actions bei Auswahl des entsprechenden Workflows zu sehen:

Pull Request

Actions-> Java CD builds jar

3.8.2 Jar-Datei herunterladen

Die Jar-Datei befindet sich als Artefakt im jeweiligen RUN des Java CD builds jar-Workflows und kann von dort heruntergeladen werden.

Die oberste Run in der Liste beinhaltet die aktuelle Jar-Datei

Actions-> Java CD builds jar

Java CD builds jar

3 workflow runs

Run	Status	Event	Branch	Actor
task-107: merge branch "create-CD-workflow" into main	Success	task-107/create-CD-workflow	Today at 1:41 PM	...
Task 107/create cd workflow	Success	task-107/create-CD-workflow	Today at 12:10 PM	...
task-107: changed the naming of the jar-file	Failure	task-107/create-CD-workflow	Today at 11:54 AM	...

task-107: merge branch "create-CD-workflow" into main #3

Summary

Triggered via pull request 1 hour ago

Status: Success

Total duration: 20s

Artifacts: 1

cd.yml

on: pull_request

build

Artifacts

Produced during runtime

Name	Size	Digest
260116_SLM_TicTacToe_task-107-create-CD-workflow.jar	3.03 KB	sha256:9d4295315beb34c6fe17e6882b9c46633c34642edbcfa694bc1d7...

3.8.3 Jar-Datei lokal ausführen

Die heruntergeladene jar.Datei kann in der Console ausgeführt werden:

java -jar DATEINAME